UNIVERSIDAD AUTONOMA GABRIEL RENE MORENO
FACULTAD DE INGENIERIA EN CIENCIAS DE LA COMPUTACION Y TELECOMUNICACIONES



DISEÑO E IMPLEMENTACION DE UN ASISTENTE VIRTUAL PARA TIENDAS DE ROPA, MEDIANTE UN PIPELINE DE NLP, ADAPTADO AL DOMINIO Y COMPARACION DE MODELOS DE CLASIFICACION(REGRESION LOGISTICA Y NAIVE BAYES)

Integrantes	Carrera	Registro
- Aranibar Fernandez Carmen Jenifer	Ing. Informática	222007419
- Calani Garcia Fernando Javier	Ing. Informática	220000654
- Gonzales Terrazas Ramiro	Ing. Informática	981015115
- Rojas Farell Jorge Leonel	Ing. Informática	218131941

Materia : ELEC103 – Tópicos Avanzados de Programación .(Algoritmos Genéticos.)

Grupo : SB

Docente : MSc. Ing. Kawaida Villegas Kenji

Semestre : 2 – 2026

Santa Cruz de la Sierra – Bolivia
Gestión 2 – 2026

Contenido

1. Problema 1

2. Datos 1

2.1. Dataset de entrenamiento 1

2.2. Datasets complementarios (docs/datasets, fecha de obtención: 2 de noviembre de 2026) 2

3. Pipeline de NLP 2

4. Modelos (fase 6, Jorge) 3

4.1. Análisis de errores 4

5. Arquitectura 5

6. Reproducibilidad y ejecución 6

7. Limitaciones 6

8. Declaración de uso de asistentes de IA 6

9. Distribución del trabajo 7

10. Anexos 7

10.1. Anexo A. Distribución de clases del dataset (notebook 01_eda) 7

10.2. Anexo B. Longitud de las frases (notebook 01_eda) 8

10.3. Anexo C. Palabras más frecuentes sin stop words (notebook 01_eda) 8

10.4. Anexo D. F1 macro en validación cruzada: stop words ajustadas vs spaCy y lema vs stem (notebook 02b) 8

10.5. Anexo E. Matriz de confusión del modelo en producción 9

10.6. Anexo F. Aplicación: panel «Cómo se procesa el texto», fases 1 a 3 con datos reales de la API 9

10.7. Anexo G. Aplicación: stop words, lemas, intención y top 3 del modelo entrenado 10

DISEÑO E IMPLEMENTACION DE UN ASISTENTE VIRTUAL PARA TIENDAS DE ROPA, MEDIANTE UN PIPELINE DE NLP, ADAPTADO AL DOMINIO Y COMPARACION DE MODELOS DE CLASIFICACION(REGRESION LOGISTICA Y NAIVE BAYES)

1. Problema

Una tienda de ropa recibe por chat consultas repetitivas de clientes (stock, precios, tallas, envíos, devoluciones, horarios, pagos) y de su área administrativa (ventas, más vendidos, stock bajo). El sistema clasifica cada mensaje en una de 14 intenciones y responde con los datos reales de la tienda importados en una base de datos, por texto o por voz.

Tarea principal: clasificación supervisada de texto en español (14 clases). Si la confianza del modelo es menor que el umbral (0,60), el asistente responde que no entendió en lugar de inventar.

2. Datos

2.1. Dataset de entrenamiento

database/raw/intenciones.csv: 1120 frases etiquetadas, 14 intenciones con 80 frases cada una (balanceado). Columnas: texto, intencion, rol, fuente.

- Origen: frases escritas por el grupo (138, fuente = manual) y frases generadas con un asistente de IA y revisadas por el grupo (982, fuente = ia). Ver la declaración de uso de IA (sección 8).
- Fecha de construcción: octubre y noviembre de 2026. Licencia: uso académico del grupo.
- Análisis exploratorio (notebook 01_eda): sin valores faltantes ni duplicados; frases cortas (8,4 palabras en promedio, entre 2 y 16); la palabra «tienen» aparece en 10 intenciones y explica parte de las confusiones.

- División estratificada 80/20 con semilla 42: 896 frases de entrenamiento y 224 de prueba (16 por intención). El conjunto de prueba no se usa para decidir nada.
- Desbalance: no existe (80 por clase). Igual se usa F1 macro como métrica principal.

2.2. Datasets complementarios (docs/datasets, fecha de obtención: 2 de noviembre de 2026)

Dataset	Fuente	Licencia	Filas	Uso
1. Frases de atención al cliente	Bitext Customer Support, traducción al español (Hugging Face)	CDLA-Sharing-1.0	6.320	Frases reales de envíos, devoluciones y pagos; entrenamiento experimental
2. Consultas fuera de dominio	MASSIVE es-ES de Amazon Science + Bitext	CC BY 4.0 / CDLA-Sharing-1.0	25.538	Calibrar y evaluar el rechazo de consultas ajenas
3. Inventario y ventas	Amazon Reviews'23, Amazon_Fashion (McAuley Lab, UCSD; Hou et al., 2024, arXiv:2403.03952)	Uso académico citando el artículo	8.000 productos + 15.000 ventas	Importar en la aplicación; respuestas del chatbot

En el dataset 3 el stock es simulado (Amazon no lo publica) y cada compra verificada cuenta como una venta. Los nombres de producto en español se derivan del tipo de prenda, la marca y el color; el título original se conserva. El script docs/datasets/construir_datasets.py regenera los tres archivos.

3. Pipeline de NLP

Las fases 1 a 4 están en la clase TextPreprocessor (backend/src/nlp/pipeline.py), que es el primer paso del modelo guardado. El mismo objeto procesa los datos de entrenamiento y cada

consulta de la API: el backend nunca reimplementa el preprocesamiento. Todos los parámetros están en backend/src/config.yaml.

Fase	Decisión	Justificación / evidencia
1. Normalización (Carmen)	Minúsculas; se quitan tildes, puntuación, URLs, correos, menciones y emojis; se conservan la ñ, números, decimales, «s/m» y «talla-m»	Números = tallas y precios. Conservarlos o eliminarlos empata en F1 (0,841): solo el 2,7 % de las frases tiene números; se conservan por criterio del dominio (notebook 02a)
2. Tokenización (Carmen)	Expresión regular propia del dominio	Resuelve «talla-m» → talla, m; «100bs» → 100, bs; mantiene «s/m» y «12.50» (split() no alcanza)
3. Stop words (Fernando)	Lista de spaCy ajustada: se conservan 35 palabras (no, sin, más, cuánto, dónde, hay, tienen, buenas, bajo, total…) y se agregan muletillas (porfa, oye)	F1 macro CV: 0,841 con la lista ajustada contra 0,756 con la de spaCy (+8,5 puntos); mejoran sobre todo productos_mas_vendidos (+0,25) y consultar_stock (+0,20) (notebook 02b)
4. Lematización / stemming (Fernando)	Lematización con spaCy (es_core_news_sm), con una tabla de correcciones para palabras de chat («hola»→«holar», «recomiendas»→«recomiendir»)	Empate técnico: lema 0,841 y stem 0,844 (diferencia menor que la desviación ±0,02). Se elige lema por interpretabilidad (palabras reales en el frontend y en los términos influyentes)
5. N-gramas y TF-IDF (Ramiro)	ngram_range (1, 2), min_df 1, max_df 0,95, sin límite de vocabulario, sublinear_tf	Comparación unigramas vs unigramas + bigramas en el notebook 02c; los bigramas capturan «talla m», «sin stock», «más vendido»
6. Modelo (Jorge)	Regresión logística en producción, comparada con Naive Bayes y una línea base; entrenamiento reproducible con un comando	Ver sección 4 y notebook 03 (validación cruzada, análisis de errores y detector de consultas fuera de dominio)

4. Modelos (fase 6, Jorge)

Se comparan regresión logística y Naive Bayes (ComplementNB) contra una línea base que predice siempre la clase más frecuente. Hiperparámetros elegidos con GridSearchCV y validación cruzada estratificada de 5 particiones sobre el entrenamiento, con F1 macro (backend/src/models/train.py).

Modelo	Hiperparámetros	F1 macro CV	Exactitud prueba	F1 macro prueba
Línea base (clase más frecuente)	—	—	7,1 %	0,009
Naive Bayes (ComplementNB)	alpha = 2.0	0,801	77,7 %	0,774
Regresión logística (en producción)	C = 10.0, class_weight = balanced	0,841	81,7 %	0,818

Elección: la regresión logística gana en validación cruzada y en prueba. Es adecuada para texto vectorizado con TF-IDF (muchas columnas casi todas en cero), funciona bien con pocos datos, entrega probabilidades para el umbral de confianza y es interpretable: sus pesos muestran qué términos empujan hacia cada intención (panel de métricas del frontend). Naive Bayes asume independencia entre palabras y queda unos 4 puntos abajo.

4.1. Análisis de errores

- Intenciones con menor F1 en prueba: stock_bajo (0,621), consultar_stock (0,714), consultar_tallas (0,727); con mayor F1: metodos_pago (0,897), productos_mas_vendidos (0,897), saludo (0,941).
- consultar_stock se confunde con consultar_tallas cuando la frase menciona una talla («¿tienen stock de polera en talla M?»): la palabra «talla» pesa más que «stock». Se corrige con más frases de stock con talla.
- Las 4 intenciones de administración se confunden entre sí porque comparten «ventas», «dame» y nombres de prendas.

- El conjunto de prueba tiene 16 frases por intención: el resultado varía varios puntos según qué frases caen en él; por eso las decisiones se toman con validación cruzada.

Extra (Jorge): detector de consultas fuera de dominio (backend/src/models/ood.py) que compara 6 estrategias de rechazo. Calibrado con el dataset 2 (1.000 consultas reales) acepta el 72 % de las consultas válidas y rechaza el 84 % de las ajenas.

5. Arquitectura

Texto o voz (navegador) → frontend React → POST /chat (FastAPI) → modelo.joblib [fases 1-4 → TF-IDF → regresión logística] → intención y probabilidad (si < 0,60: «no entendí») → extracción de entidades (talla, color, prenda, fecha) → consulta a PostgreSQL → respuesta en texto (el navegador la lee en voz alta).

Capa	Tecnología	Contenido
Datos	PostgreSQL 16 (contenedor db)	Tablas productos, ventas y tienda; importación masiva de CSV/Excel
NLP y modelos	spaCy, NLTK, scikit-learn	backend/src: pipeline, entrenamiento con un comando, artefactos
Backend	FastAPI (validación con Pydantic y documentación automática en /docs)	/health, /predict, /predict/lote, /chat, /metrics, /reportes/{tipo}, /inventario/importar, /metricas/entrenamiento
Frontend	React 18 + Vite, servido con Nginx	Chat con voz, vista del preprocesamiento paso a paso, apartado de inventario y métricas con comparación de modelos y matriz de confusión

Validaciones de la API: texto vacío o de más de 500 caracteres → 422; archivo con formato incorrecto → 400; mensajes en español. Pruebas automatizadas con pytest: 136 pruebas (fases de NLP, API, inventario, respuestas).

6. Reproducibilidad y ejecución

- Sistema completo: docker compose up --build (frontend en localhost:5173, API en localhost:8000/docs).
- Entrenamiento desde los datos crudos con un solo comando: python -m src.models.train (genera artifacts/modelo.joblib, ood_detector.joblib y metricas.json; si falta el modelo, el contenedor lo entrena al arrancar).
- Dependencias con versiones fijadas (backend/requirements.txt) y semilla 42 en división, validación y modelos.
- Modificaciones en vivo: cambiar nlp.modo, stopwords_extra, ngram_range o C en config.yaml y reentrenar.

7. Limitaciones

- El 88 % del dataset de entrenamiento se generó con IA; las frases reales de fuentes públicas (Bitext) cubren solo 3 intenciones.
- Con 80 frases por intención, el F1 de prueba varía varios puntos según la partición.
- El inventario de demostración usa stock simulado y casi no trae tallas por producto.
- El detector de consultas ajenas no está conectado al endpoint /chat, que usa solo el umbral de probabilidad.

8. Declaración de uso de asistentes de IA

Conforme a la sección 10 del enunciado, el equipo declara que usó asistentes de IA para: generar y parafrasear parte de las frases del dataset de intenciones (fuente = ia), apoyar la escritura y revisión de código, pruebas y notebooks, y redactar documentación. Todo el contenido fue revisado por el grupo, y cada integrante puede explicar y modificar el código que subió. Los datos externos se citan con su fuente y licencia en la sección 2.2.

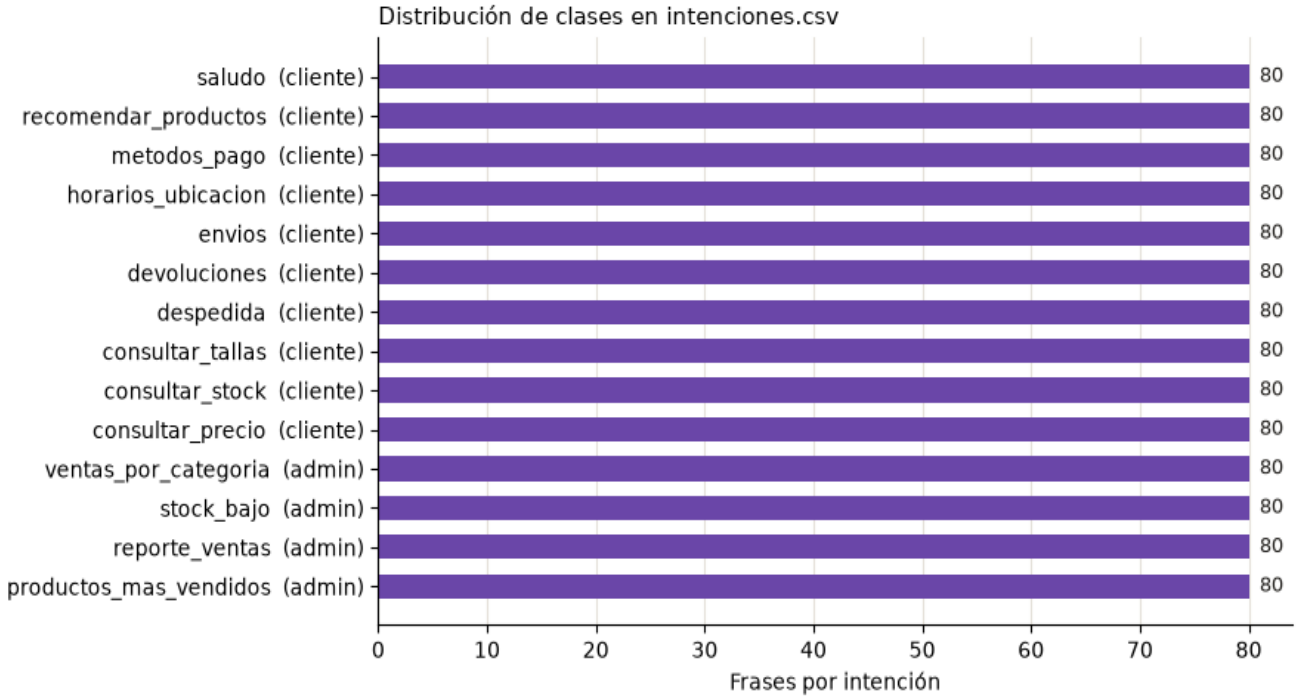
9. Distribución del trabajo

Integrante	Fases de NLP / ML	Capa del sistema
Carmen	1. Normalización · 2. Tokenización (notebook 02a)	Frontend base (React)
Fernando	3. Stop words · 4. Lematización y stemming · TextPreprocessor (notebook 02b)	Docker y PostgreSQL, inventario y métricas, chat con datos reales, EDA (01) y datasets
Ramiro	5. TF-IDF y n-gramas · extracción de entidades (notebook 02c)	Backend: API FastAPI y pruebas
Jorge	6. Modelos, validación cruzada, análisis de errores y detector OOD (notebook 03)	Pipeline de entrenamiento reproducible y artefactos

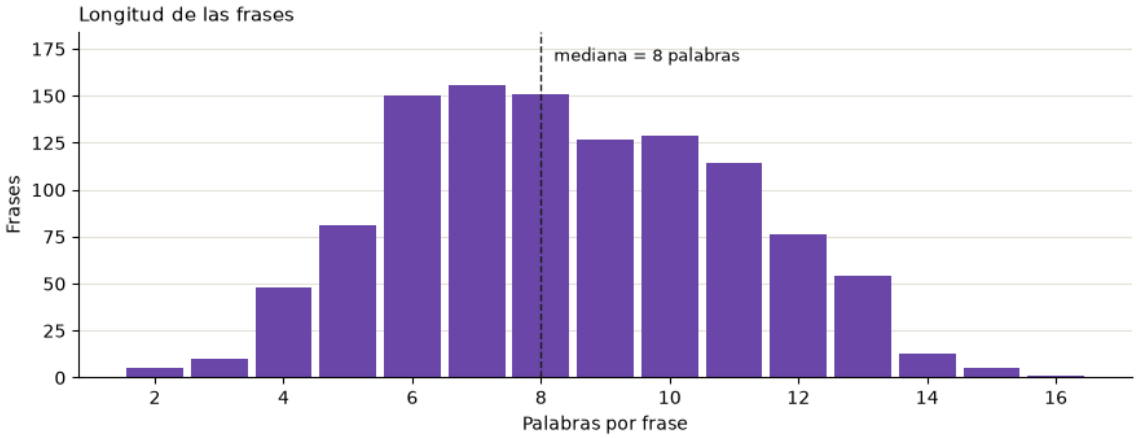
El detalle por commit está en CONTRIBUCIONES.md y en el historial del repositorio.

10. Anexos

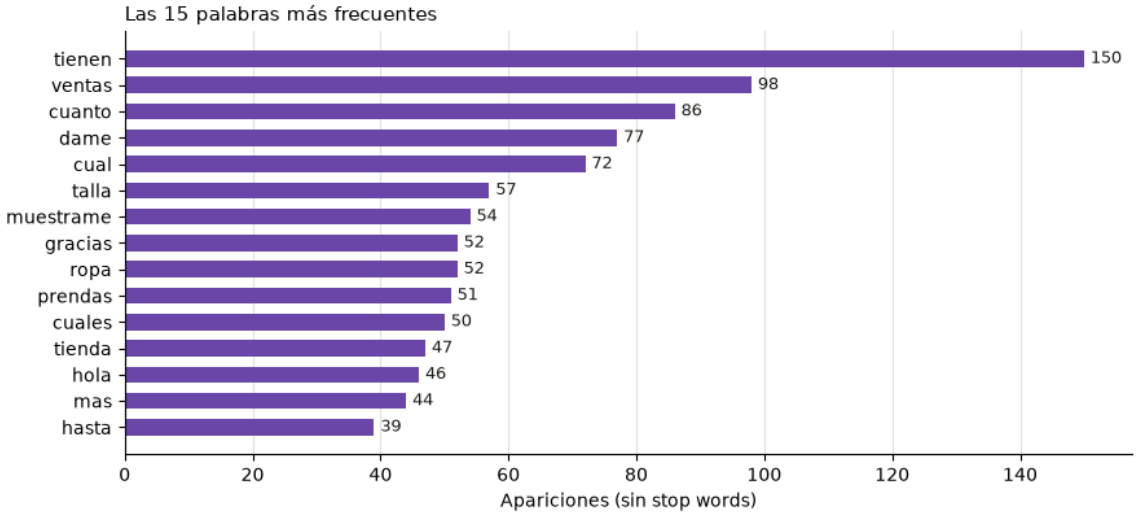
10.1. Anexo A. Distribución de clases del dataset (notebook 01_eda)



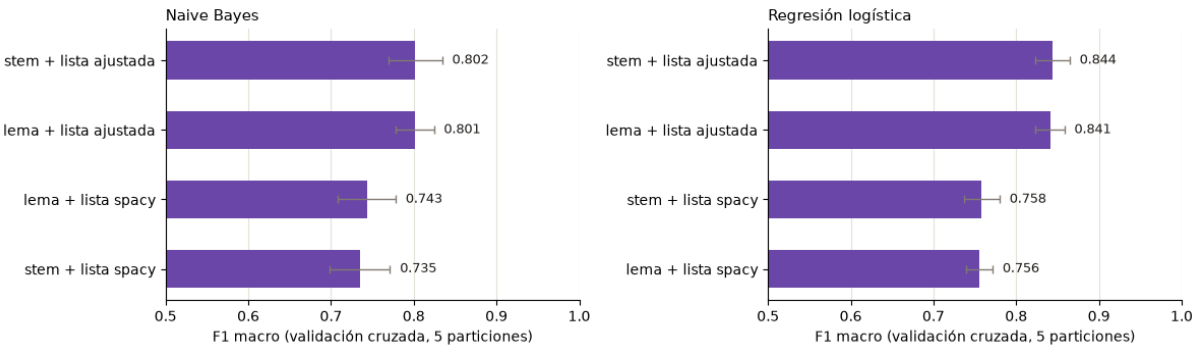
10.2. Anexo B. Longitud de las frases (notebook 01_eda)



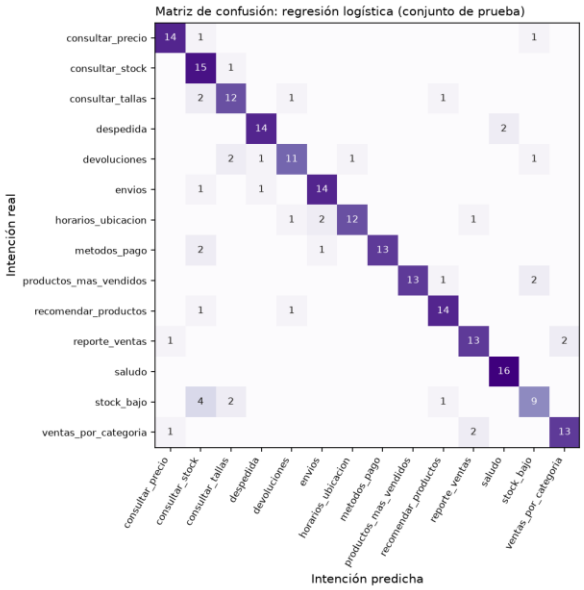
10.3. Anexo C. Palabras más frecuentes sin stop words (notebook 01_eda)



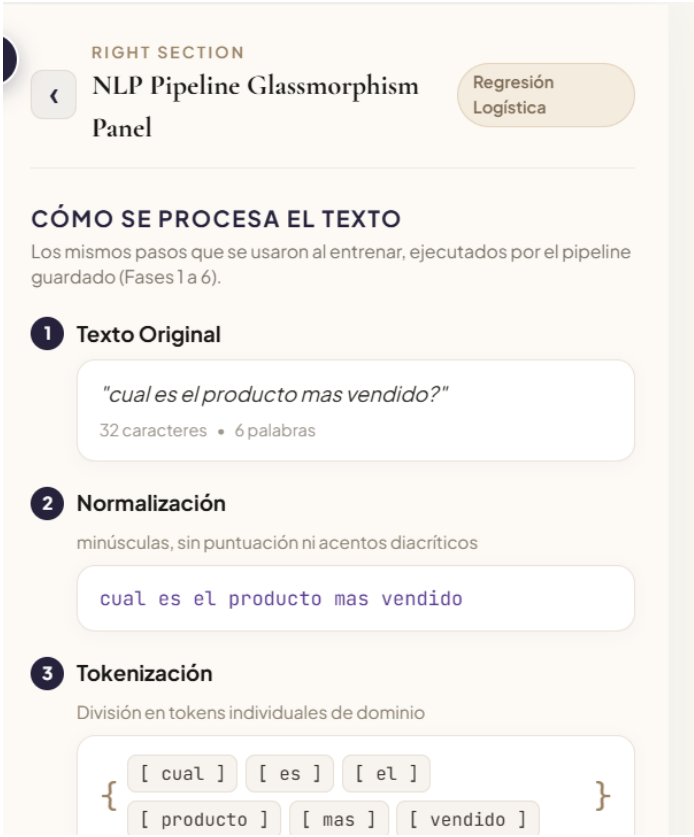
10.4. Anexo D. F1 macro en validación cruzada: stop words ajustadas vs spaCy y lema vs stem (notebook 02b)



10.5. Anexo E. Matriz de confusión del modelo en producción



10.6. Anexo F. Aplicación: panel «Cómo se procesa el texto», fases 1 a 3 con datos reales de la API



10.7. Anexo G. Aplicación: stop words, lemas, intención y top 3 del modelo entrenado

